

Installing cyipopt with HSL on Windows

Step-by-step guide for conda, MSYS2, Coin-HSL, METIS, and Pyomo/IPOPT

Last updated: June 25, 2026

Purpose. Use this guide when conda-forge cyipopt/IPOPT is installed successfully, but IPOPT fails because it is trying to use the HSL MA57 linear solver and cannot load libhsl.dll. The guide also covers the optional libmetis.dll warning that appears after cyipopt starts working.

1. What this fixes

After installing cyipopt with conda, IPOPT may fail with this message:

```
Exception of type: DYNAMIC_LIBRARY_FAILURE  
Error 126 while loading DLL libhsl.dll via LoadLibraryA  
EXIT: Library loading failure.
```

This is not usually a Pyomo model problem. It means IPOPT is being told to use the HSL MA57 linear solver, but Windows cannot find the HSL dynamic library, libhsl.dll. Once libhsl.dll is available, a second nonfatal warning may appear if libmetis.dll is missing:

```
Failed to load 'libmetis.dll' - using fallback AMD code
```

The fallback AMD message is a warning. MA57 can still run, but METIS ordering is not being used.

2. Important licensing note

- Coin-HSL/HSL is license-controlled software. Do not commit Coin-HSL source files, libhsl.dll, libmetis.dll, or other HSL binaries to a public or private GitHub repository unless your license explicitly permits that distribution.
- Each user should obtain Coin-HSL from the official HSL/STFC licensing page and comply with the license terms, including citation requirements.
- The conda-forge cyipopt/IPOPT installation does not include HSL/MA57 binaries by default.

3. Basic cyipopt install

Use Anaconda Prompt or another Command Prompt where conda is available:

```
conda activate your-env
conda install -c conda-forge cyipopt
```

Replace `your-env` with your own conda environment name.

4. Full MA57 setup overview

1. Install cyipopt from conda-forge.
2. Download Coin-HSL from the official HSL/STFC page.
3. Install MSYS2 build tools and use the MSYS2 MinGW x64 shell.
4. Build libcoinhsl-0.dll from Coin-HSL source and copy it into the conda environment as libhsl.dll.
5. Copy required MinGW runtime DLLs into the same conda environment folder if Windows cannot load libhsl.dll.
6. Optionally build libmetis-0.dll and copy it into the conda environment as libmetis.dll to remove the METIS warning.

5. Download Coin-HSL

Download Coin-HSL from the official HSL page for IPOPT:

<https://www.hsl.rl.ac.uk/ipopt/>

After extraction, the source folder should look similar to this:

```
coinhsl-2019.05.21/
  configure
  Makefile.in
  common/
  loadmetis/
  ma57/
  hsl_ma77/
  hsl_ma86/
  hsl_ma97/
```

If you downloaded a Windows binary package instead of source, it may already contain a bin folder with DLLs. In that case, copy the DLLs from that bin folder into %CONDA_PREFIX%\Library\bin and skip the source build sections.

6. Install MSYS2 build tools

Install MSYS2 from <https://www.msys2.org/>. Then open MSYS2 MinGW x64 from the Start menu. The prompt must say MINGW64.

Do not use the UCRT64 shell with the mingw-w64-x86_64 packages below. If the prompt says UCRT64, either open MSYS2 MinGW x64 or install the UCRT equivalents instead.

Install the build tools:

```
pacman -Syu
pacman -S --needed base-devel autoconf automake libtool mingw-w64-x86_64-gcc-fortran mingw-w64-x86_64-openblas
```

Verify that gfortran is visible in the MINGW64 shell:

```
which gfortran
gfortran --version
```

Expected location:

```
/mingw64/bin/gfortran
```

7. Navigate to the Coin-HSL source folder

In MSYS2 shells, use Unix-style paths. For example:

```
cd /c/Users/surname/Downloads/coinhsl-2019.05.21/coinhsl-2019.05.21
```

This Windows-style path will not work directly inside MSYS2 bash:

```
cd C:\Users\surname\Downloads\coinhsl-2019.05.21\coinhsl-2019.05.21
```

8. Build libhsl.dll from Coin-HSL source

From the Coin-HSL source folder in MSYS2 MinGW x64, run:

```
autoreconf -fi
```

Patch the wrapper symbol name. This fixes the link error involving metis_nodend_:

```
sed -i 's/void METIS_NODEND(/void metis_nodend_(/' loadmetis/loadmetis.c
```

Configure Coin-HSL against the MinGW OpenBLAS library:

```
./configure --with-blas="-L/mingw64/lib -lopenblas"
```

Build the HSL library:

```
make libcoinhsl.la
```

Check for the output DLL:

```
ls .libs/*.dll
```

Expected output includes (don't need to be exact):

```
.libs/libcoinhsl-0.dll
```

9. Copy libhsl.dll into the conda environment

Switch to Anaconda Prompt. Activate the same environment where cyipopt is installed:

```
conda activate idaes-pse
```

Copy and rename the Coin-HSL DLL so IPOPT can find the exact filename it expects:

```
copy /Y "C:\Users\lilon\Downloads\coinhsl-2019.05.21\coinhsl-2019.05.21\.libs\libcoinhsl-0.dll" "%CONDA_PREFIX%\Library\bin\libhsl.dll"
```

Verify:

```
dir "%CONDA_PREFIX%\Library\bin\libhsl.dll"
```

10. Copy required MinGW runtime DLLs

If IPOPT still reports Error 126 after libhsl.dll is copied, Windows may be finding libhsl.dll but missing one of its dependent runtime DLLs. Copy these into the same conda folder:

```
copy /Y "C:\msys64\mingw64\bin\libgfortran-*.dll" "%CONDA_PREFIX%\Library\bin\"
copy /Y "C:\msys64\mingw64\bin\libquadmath-*.dll" "%CONDA_PREFIX%\Library\bin\"
copy /Y "C:\msys64\mingw64\bin\libgcc_s_seh-1.dll" "%CONDA_PREFIX%\Library\bin\"
copy /Y "C:\msys64\mingw64\bin\libwinpthread-1.dll" "%CONDA_PREFIX%\Library\bin\"
copy /Y "C:\msys64\mingw64\bin\*openblas*.dll" "%CONDA_PREFIX%\Library\bin\"
```

The OpenBLAS DLL name can vary. To inspect the exact file name first:

```
dir C:\msys64\mingw64\bin\*openblas*.dll
```

11. Remove the libmetis.dll warning

If MA57 works but IPOPT prints this warning, METIS is missing:

```
Failed to load 'libmetis.dll' - using fallback AMD code
```

This is not fatal. To remove the warning, build and copy libmetis.dll.

11.1 Download METIS 4.0.3

In MSYS2 MinGW x64, from the Coin-HSL source folder:

```
cd /c/Users/lilon/Downloads/coinhsl-2019.05.21/coinhsl-2019.05.21
curl -L -O http://coin-or-tools.github.io/ThirdParty-Metis/metis-4.0.3.tar.gz
tar -xzf metis-4.0.3.tar.gz
ls -d metis-4.0.3
```

The Coin-HSL source folder should contain metis-4.0.3 directly:

```
coinhsl-2019.05.21/
metis-4.0.3/
configure
loadmetis/
ma57/
```

11.2 Patch old METIS 4 source issues

Newer GCC versions are stricter than the old METIS 4 source expects. Apply these patches:

```
cp metis-4.0.3/Lib/proto.h metis-4.0.3/Lib/proto.h.bak
sed -i 's@/\*void GKfree(void \*\*,\.\.\.); \*/@void GKfree(void **,...);@' metis-4.0.3/Lib/proto.h
sed -i 's/SelectQueueoneWay/SelectQueueOneWay/g' metis-4.0.3/Lib/proto.h
sed -i 's/SelectQueueoneWay/SelectQueueOneWay/g' metis-4.0.3/Lib/rename.h
```

Build METIS with a compatibility warning flag:

```
make libmetis.la CFLAGS="-g -O2 -Wno-incompatible-pointer-types"
```

If the compiler stops on an implicit declaration error, use the broader compatibility command:

```
make libmetis.la CFLAGS="-g -O2 -Wno-incompatible-pointer-types -Wno-implicit-function-declaration"
```

Check for the DLL:

```
ls .libs/*metis*.dll
```

Expected output:

```
.libs/libmetis-0.dll
```

11.3 Copy libmetis.dll into the conda environment

In Anaconda Prompt:

```
conda activate idaes-pse
copy /Y "C:\Users\lilon\Downloads\coinhsl-2019.05.21\coinhsl-2019.05.21\.libs\libmetis-0.dll" "%CONDA_PREFIX%\Library\bin\libmetis.dll"
```

Verify:

```
dir "%CONDA_PREFIX%\Library\bin\libmetis.dll"
```

12. Verification checklist

- Anaconda Prompt is using the expected conda environment: `conda activate your-env`
- `libhsl.dll` exists in `%CONDA_PREFIX%\Library\bin`
- If using METIS, `libmetis.dll` exists in `%CONDA_PREFIX%\Library\bin`
- Required MinGW runtime DLLs are in `%CONDA_PREFIX%\Library\bin` if Error 126 persists

```
conda activate your-env
dir "%CONDA_PREFIX%\Library\bin\libhsl.dll"
dir "%CONDA_PREFIX%\Library\bin\libmetis.dll"
```

13. Troubleshooting

Symptom	Likely cause	Fix
configure: Fortran compiler cannot create executables	You are in the wrong MSYS2 shell or gfortran is not installed.	Use MSYS2 MinGW x64. Install mingw-w64-x86_64-gcc-fortran and confirm which gfortran returns <code>/mingw64/bin/gfortran</code> .
make: No targets specified and no makefile found	configure failed, so no Makefile was created.	Fix the configure error first, then rerun <code>./configure</code> and <code>make</code> .
autoheader: command not found	Autoconf tools are missing.	Run <code>pacman -S --needed autoconf automake libtool</code> , then <code>autoreconf -fi</code> .
undefined reference to <code>metis_nodend_</code>	The Coin-HSL METIS wrapper symbol name does not match what Fortran expects.	Patch <code>loadmetis/loadmetis.c</code> so <code>METIS_NODEND</code> becomes <code>metis_nodend_</code> , then rebuild.
Error 126 while loading DLL <code>libhsl.dll</code>	<code>libhsl.dll</code> is missing or one of its dependent DLLs is missing.	Copy <code>libcoinhsl-0.dll</code> as <code>libhsl.dll</code> into <code>%CONDA_PREFIX%\Library\bin</code> , then copy the MinGW runtime DLLs if needed.
Failed to load 'libmetis.dll' - using fallback AMD code	MA57 is working, but METIS ordering is unavailable.	Ignore it if performance is acceptable, or build <code>libmetis.la</code> and copy <code>libmetis-0.dll</code> as <code>libmetis.dll</code> .
MSYS2 <code>cd C:\Users\...</code> fails	MSYS2 bash expects Unix-style paths.	Use <code>/c/Users/lilon/...</code> instead of <code>C:\Users\lilon\...</code>

References

- cyipopt installation documentation: <https://cyipopt.readthedocs.io/stable/install.html>
- HSL for IPOPT download page: <https://www.hsl.rl.ac.uk/ipopt/>
- COIN-OR ThirdParty-Metis repository: <https://github.com/coin-or-tools/ThirdParty-Metis>
- METIS 4.0.3 archive used by COIN-OR ThirdParty-Metis: <http://coin-or-tools.github.io/ThirdParty-Metis/metis-4.0.3.tar.gz>